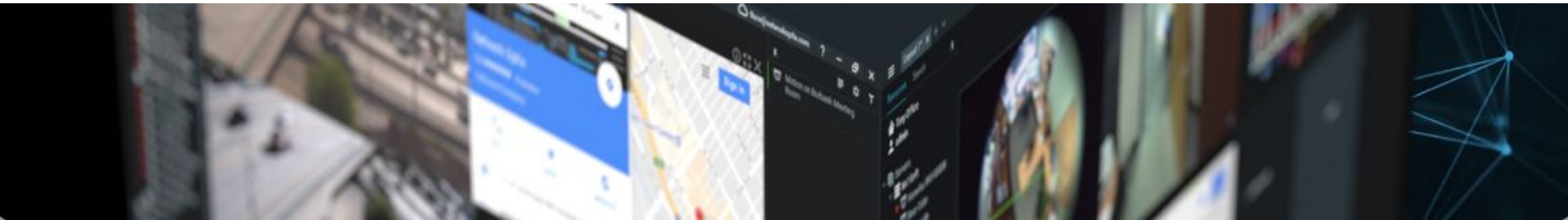




NetworkOptix

November 2025
Secure SDLC & Threat Modeling



Agenda

01

SDLC Overview

Understanding where security fits in the development lifecycle

03

Data Flow Diagrams

Visualizing system architecture and data movement

05

STRIDE Framework

Practical threat identification with real examples

02

Threat Modeling Fundamentals

Core concepts and why it matters for your team

04

Trust Boundaries

Identifying critical security checkpoints

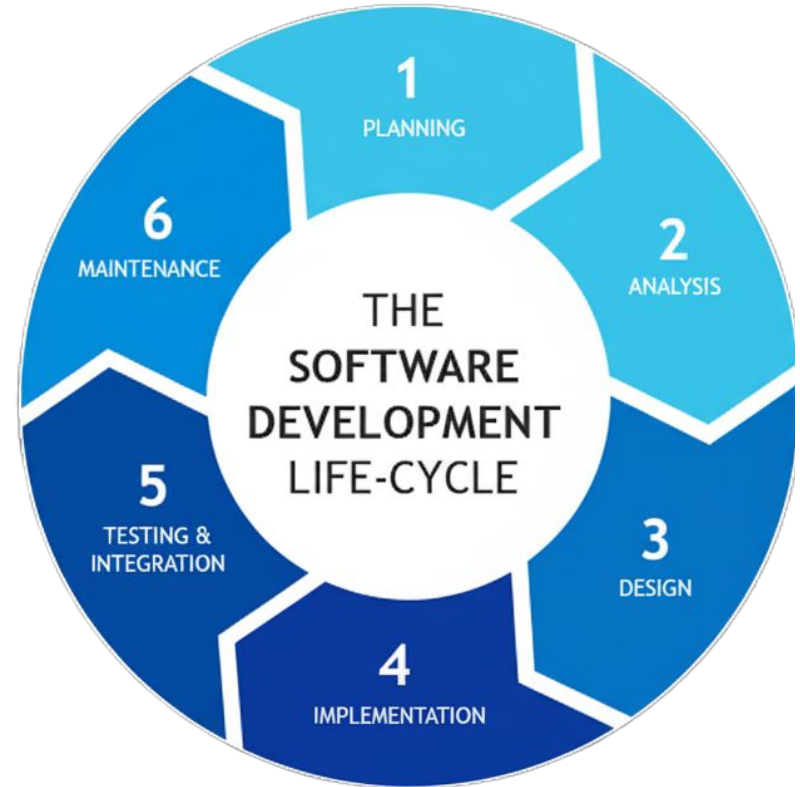
06

Wrap-up & Q&A

Key takeaways and your questions

What is SDLC

Software Development Life Cycle - is the process of initiating, developing, and maintaining an information system, encompassing a set of activities that ensure that security is considered throughout the system's life.



Security Across SDLC

Security must be integrated at every stage of development. Each phase has unique security activities that build upon one another to create resilient systems.



Requirements

Define security requirements, compliance constraints, and threat assumptions early



Design

Threat modeling sessions to identify risks before code is written



Implementation

Secure coding practices, static analysis, and vulnerability scanning



Testing

Negative testing, dynamic analysis (DAST), and fuzzing techniques



Operations

System hardening, continuous monitoring, and incident response

 **Today's focus:** We're diving deep into the Design phase, specifically threat modeling — a critical step that shapes everything downstream.

Threat Modeling Basics



What is Threat Modeling?

Threat modeling is a structured, collaborative process used to identify, evaluate, and prioritize potential security threats to a system.

Why it matters:

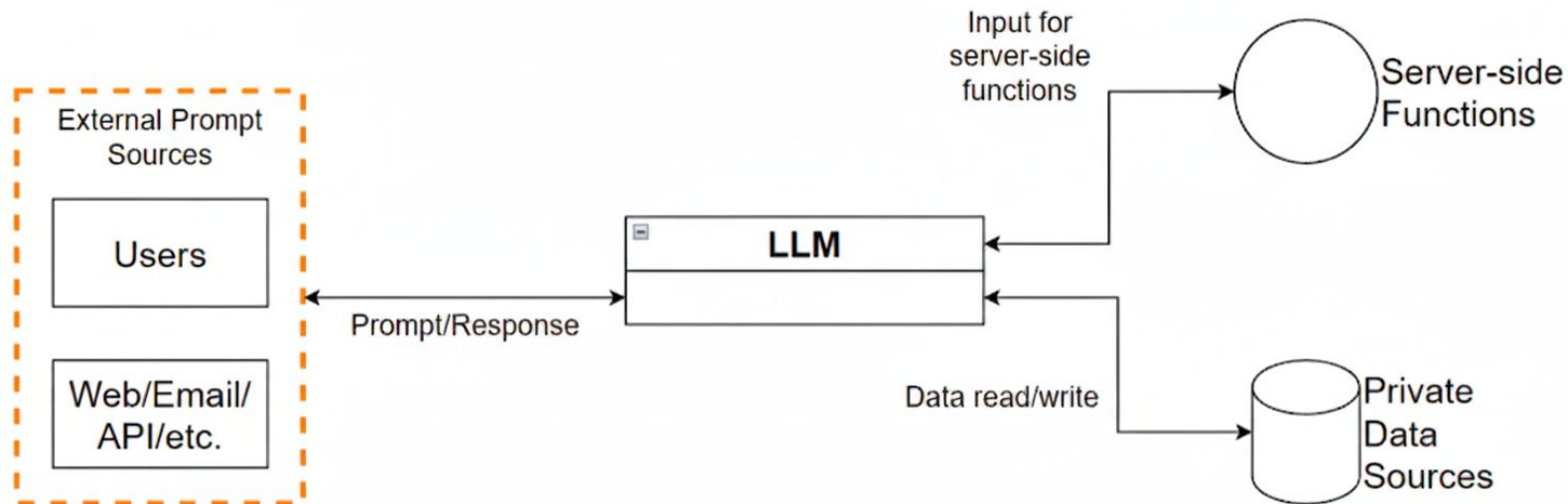
- Catches vulnerabilities before they become expensive fixes
- Aligns security efforts with business priorities
- Creates a shared security language across teams

Today's Hands-On Exercise

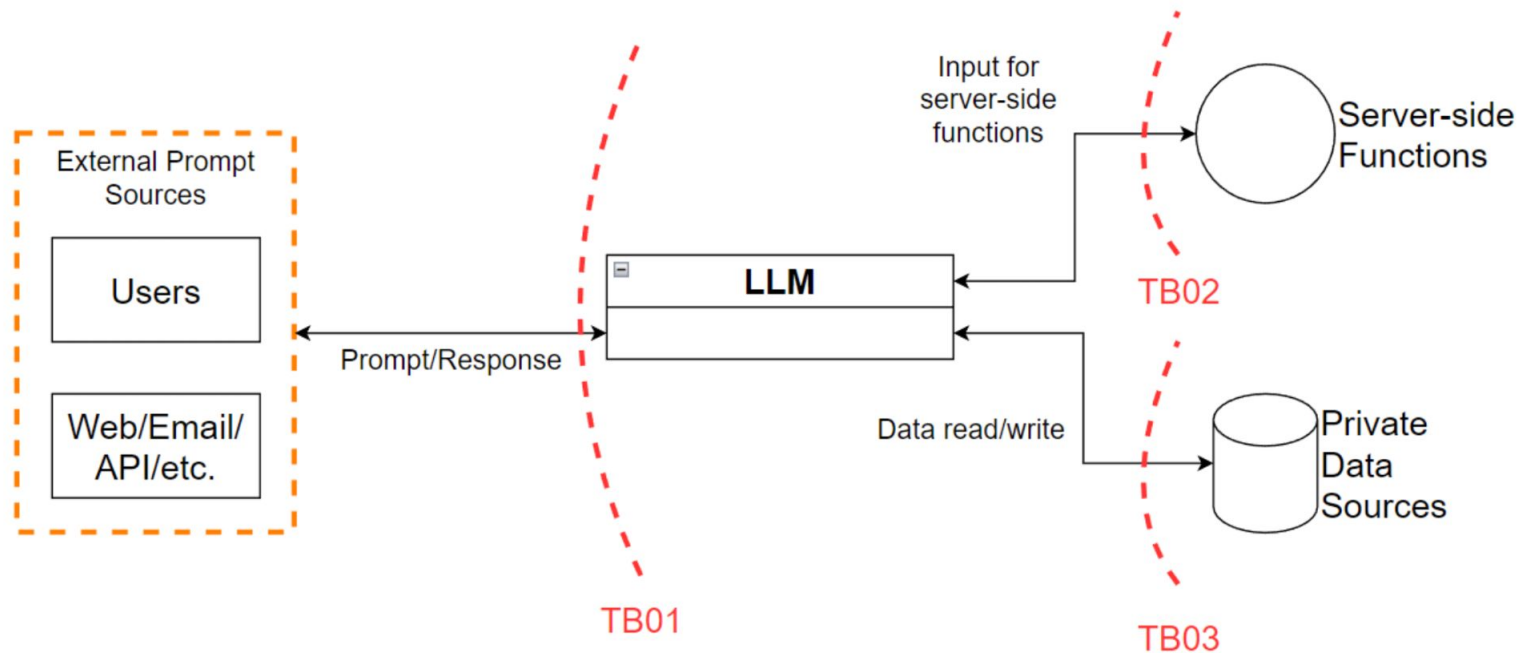
We'll build a simple threat model for a Railway Reservation System:

- **Build a Data Flow Diagram**
Map components and data flows
- **Identify Trust Boundaries**
Mark security transition points
- **Apply STRIDE**
Systematically find threats

Data flow diagram



Trust boundaries



STRIDE

STRIDE is a mnemonic framework developed by Microsoft to systematically identify security threats. Each letter represents a distinct category of threat that can compromise your system.

Spoofing

Pretending to be someone or something else to gain unauthorized access

Tampering

Maliciously modifying data or code without authorization

Repudiation

Denying that you performed an action without a way to prove otherwise

Information Disclosure

Exposing sensitive information to unauthorized parties

Denial of Service

Making a system unavailable to legitimate users

Elevation of Privilege

Gaining capabilities or access beyond what was authorized

STRIDE TB1 External endpoints

TB01	External Endpoints	
	Strengths	Weaknesses
Spoofing		VULN01: Modify System prompt (prompt injection)
Tampering		VULN02: Modify LLM parameters (temperature, length, model, etc.)
Repudiation	Proper authentication and authorization (assumed)	
Information Disclosure		VULN03: Input sensitive information to a third-party site (user behavior)
Denial of Service		
Elevation of Privilege	Proper authentication and authorization (assumed)	

LLM		
	Strengths	Weaknesses
Spoofing		
Tampering		
Repudiation		
Information Disclosure		VULN04: LLMs are unable to filter sensitive information (open research)
Denial of Service		
Elevation of Privilege		

From Threats to Action

Threat modeling only delivers value when insights translate into tangible improvements across every role. Here's how different team members turn threat models into everyday security wins.



Developers

Anticipate attack vectors early, build safer APIs with input validation, and implement defense-in-depth patterns proactively.



Product Managers

Prioritize mitigation work by balancing business risk, user impact, and resource constraints using threat severity data.



QA Engineers

Transform identified threats into realistic negative test cases, abuse scenarios, and security-focused regression suites.



Architects

Validate design resilience against identified threats, fix weak architectural links, and evolve patterns based on findings.

 **The goal:** Turn threat models into everyday design decisions — not just diagrams that gather dust.

Other Frameworks



PASTA

Process for Attack Simulation and Threat Analysis — A risk-centric, seven-stage methodology that aligns threat modeling with business objectives and asset prioritization. Ideal for enterprise environments with formal risk management.



OCTAVE

Operationally Critical Threat, Asset, and Vulnerability Evaluation — Focuses on organizational assets, critical business processes, and resilience. Well-suited for assessing risk across entire business units.



VAST

Visual, Agile, and Simple Threat modeling — Scalable and designed for agile teams, VAST distributes threat modeling activities across development sprints and multiple teams simultaneously.

Most teams start simple with STRIDE, then gradually blend in elements from these frameworks as their security programs mature and organizational needs evolve.

Key Takeaways



Security starts at design

Threat modeling helps us discover and fix security issues before they reach production, saving time and reducing risk exponentially.



Do it together — include security team early

Collaboration between developers, QA engineers, architects, product managers, and security teams transforms theoretical threats into real organizational resilience.



Make it continuous

Model → Act → Learn → Repeat.
Security works best as an ongoing habit integrated into your workflow, not a one-time checklist to complete.

Thank you!